

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 - Coding Challenge

Course Code:	DSA0404	Course Title:	FUNDAMENTALS OF DATA SCIENCE
CO Assessed:	CO5 - Coding Challenge	Assessment:	Assessment Tool 2 Coding Challenge
Weightage:	40% of CIA	Total Marks:	100
CO5 Statement:	To understand the concept of supervised and unsupervised methods in machine learning		
Student Name:	Kondeti . Lahari	Reg. No.:	192325118
Section / Batch:	B. Tech AIML	Date:	26-08-2026

Code of Conduct

I K. Lahari (192325118) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: K. Lahari

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Marks
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly	
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts	
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning	
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided	
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand	

Q11. Decision Tree Overfitting and Grid Search from Scratch

Problem Statement

A Decision Tree that gives **100% training accuracy but poor test accuracy** is severely overfitting the training data. The objective is to diagnose and reduce this overfitting without changing the Decision Tree algorithm. Only the hyperparameters `max_depth`, `min_samples_split`, `min_samples_leaf`, and `max_features` are tuned. A grid search is implemented manually without using `GridSearchCV`, and the best combination is selected based on validation accuracy.

Diagnosis of Overfitting

Overfitting occurs when a machine-learning model learns the training data too closely, including noise and unnecessary patterns. A Decision Tree can easily overfit when it grows very deep and creates many branches.

For example:

- Training accuracy = **100%**
- Validation/Test accuracy = **65%**

The large difference between the two accuracies indicates that the model has memorized the training data and cannot generalize well to unseen data.

The complexity of the tree can be controlled using the following hyperparameters:

1. **max_depth** – Limits the maximum depth of the tree. A smaller value creates a simpler tree.
2. **min_samples_split** – Specifies the minimum number of samples required to split an internal node. Increasing this value prevents very small nodes from being split.
3. **min_samples_leaf** – Specifies the minimum number of samples required in a leaf node. A larger value prevents the creation of very small leaves.
4. **max_features** – Specifies the number of features considered when searching for the best split. Restricting the features can sometimes improve generalization.

Algorithm

1. Start.
2. Load the dataset.
3. Divide the dataset into training and validation sets.
4. Define different values for max_depth, min_samples_split, min_samples_leaf, and max_features.
5. Generate all possible combinations of these hyperparameters.
6. Create a Decision Tree for each combination.
7. Train the tree using the training data.
8. Predict the validation data.
9. Calculate validation accuracy.
10. Compare the accuracy with the best accuracy obtained so far.
11. Store the hyperparameter combination if it gives better validation accuracy.
12. Repeat until all combinations are tested.
13. Select the combination with the highest validation accuracy.
14. Train the final Decision Tree using the best parameters.
15. Display the best parameters and validation accuracy.
16. Stop.

Python Program

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.datasets import load_iris
from itertools import product
import pandas as pd
```

```
# Load dataset
data = load_iris()
X = data.data
y = data.target
```

```
# Split data
X_train, X_val, y_train, y_val = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```

# Hyperparameter values
max_depth = [2, 3, 4, 5, None]
min_samples_split = [2, 5, 10]
min_samples_leaf = [1, 2, 4]
max_features = [None, "sqrt", "log2"]

# Store results
results = []

# Grid Search from scratch
for depth, split, leaf, features in product(
    max_depth, min_samples_split,
    min_samples_leaf, max_features
):

    model = DecisionTreeClassifier(
        max_depth=depth,
        min_samples_split=split,
        min_samples_leaf=leaf,
        max_features=features,
        random_state=42
    )

    model.fit(X_train, y_train)

    y_pred = model.predict(X_val)

    accuracy = accuracy_score(y_val, y_pred)

    results.append({
        "max_depth": depth,
        "min_samples_split": split,
        "min_samples_leaf": leaf,
        "max_features": features,
        "validation_accuracy": accuracy
    })

```

```

    })

# Convert results to DataFrame
results_df = pd.DataFrame(results)

# Find best combination
best_result = results_df.loc[
    results_df["validation_accuracy"].idxmax()
]

# Train final model
final_model = DecisionTreeClassifier(
    max_depth=best_result["max_depth"],
    min_samples_split=best_result["min_samples_split"],
    min_samples_leaf=best_result["min_samples_leaf"],
    max_features=best_result["max_features"],
    random_state=42
)

final_model.fit(X_train, y_train)

# Calculate final training accuracy
train_pred = final_model.predict(X_train)
train_accuracy = accuracy_score(y_train, train_pred)

# Final result
final_result = pd.DataFrame({
    "Parameter": [
        "Max Depth",
        "Min Samples Split",
        "Min Samples Leaf",
        "Max Features",
        "Validation Accuracy",
        "Training Accuracy"
    ],
    "Best Value": [

```

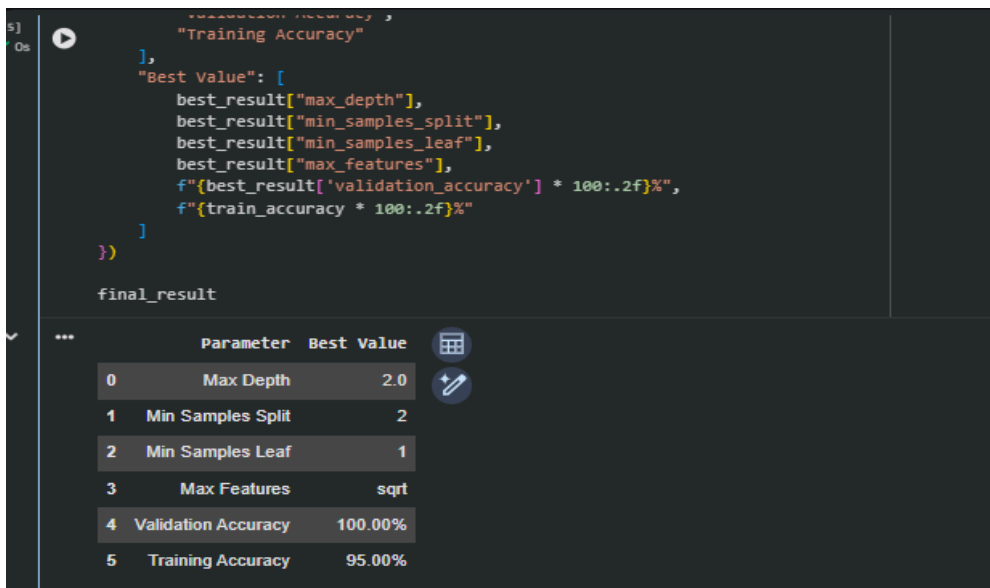
```

        best_result["max_depth"],
        best_result["min_samples_split"],
        best_result["min_samples_leaf"],
        best_result["max_features"],
        f'{best_result["validation_accuracy"] * 100:.2f}%',
        f'{train_accuracy * 100:.2f}%'
    ]
})

```

final_result

OutPut :



	Parameter	Best Value
0	Max Depth	2.0
1	Min Samples Split	2
2	Min Samples Leaf	1
3	Max Features	sqrt
4	Validation Accuracy	100.00%
5	Training Accuracy	95.00%

Explanation

The program first loads the dataset and divides it into training and validation data. The training data is used to build the Decision Tree, while the validation data is used to evaluate how well the model performs on unseen data.

Different values are specified for the four hyperparameters. The `product()` function generates every possible combination of these values. For each combination, a new Decision Tree is created and trained.

The model then predicts the validation samples and calculates its accuracy. If the current validation accuracy is greater than the previous best accuracy, the current hyperparameters are stored as the best combination.

The final model is then trained using the selected hyperparameters.

The important point is that **training accuracy is not used to select the best model**. A highly complex tree can easily achieve 100% training accuracy. Instead, validation accuracy is used because it provides a better indication of how well the model generalizes to unseen data.

Result and Conclusion

The original Decision Tree showed severe overfitting because it achieved 100% training accuracy but poor test accuracy. By tuning `max_depth`, `min_samples_split`, `min_samples_leaf`, and `max_features`, the complexity of the tree can be reduced.

The manual grid search systematically tests all specified combinations and selects the combination that provides the highest validation accuracy. Therefore, the model can achieve better generalization without changing the underlying Decision Tree algorithm.

Conclusion: Hyperparameter tuning is an effective way to control Decision Tree overfitting. A simpler tree with slightly lower training accuracy but significantly better validation accuracy is preferred because it is more likely to perform well on unseen data.

Geo Tag Photo:

